

QUE · STAKEHOLDER & END-USER BRIEF

Problems We Solve + Complete User Guide

Product: **Que** (SchemaGraph) — schema + HITL decision + attested delivery into *your* systems

Codebase: `adc/schemagraph` · App `http://localhost:5173` · API `http://localhost:8787`

Document version: **1.0** · Date: 5 Aug 2026 · Audience: buyers, pilots, end users, demos

One-liner: Que is the schema + human-in-the-loop decision + attested delivery plane into customer systems — *not* where the cleaned data lake lives.

This PDF combines (1) real-world problems Que solves today, (2) how the product actually works (roles, sync, joins, samples, jobs, handoff), and (3) a screen-by-screen user guide — every major button, filter, and action.

Table of contents

1. Part A — Real-world problems Que solves now
2. Part B — How Que works (product facts)
3. Part B.1 — Roles: Viewer / Member / Admin / Owner
4. Part B.2 — Sync, joins, samples, jobs, schedules
5. Part B.3 — Explicitly out of v1
6. Part C — Complete user guide (every screen)
7. C.1 Login & account
8. C.2 App chrome & navigation
9. C.3 Workspace (schema canvas)
10. C.4 Sources (connectors)
11. C.5 Join Review + mapping assist
12. C.6 AI Chat
13. C.7 Jobs (notebook → deploy)
14. C.8 Lineage
15. C.9 Settings (people, policy, ops, billing)
16. C.10 Attestation verify (public)
17. Part D — Typical end-to-end journey
18. Part E — What's newly built (Wave 4 features)

Part A — Real-world problems Que solves now

1. “We can’t safely stitch data across systems”

Teams have Postgres + Mongo + Excel + Databricks / Snowflake / BigQuery / Salesforce with **no trusted join map**. Wrong keys silently poison pipelines.

What Que does: Syncs *schemas only*, suggests joins with scores/evidence, and forces human **Promote** before those joins become job truth.

2. “Catalogs inventory; nobody ships a correct stitch job”

Catalog tools describe what exists. Data engineers still hand-write joins and hope.

What Que does: Reviewed schema + joins → **frozen contract** → notebook job → **attested export** / optional CTAS or VIEW in *their* warehouse — something a DE can actually use.

3. “Security won’t let AI (or a vendor) touch our lake”

CISOs block “send us the lake” for AI copilots and SaaS vendors.

What Que does: Schema-first + optional **capped / scrubbed samples**. Assist stays off full row dumps. That unblocks evaluation and enterprise conversation.

4. “Who changed what, and can we prove this export?”

Multi-person workspaces need accountability for diligence and compliance.

What Que does: Roles (viewer / member / admin / owner), audit log, drift alerts, and HMAC-attested exports so “this stitch was reviewed and frozen” is defensible.

5. “Schema drifts and yesterday’s joins break silently”

After upstream schema change, old joins keep shipping until someone notices in production.

What Que does: Surfaces drift after sync; policy can **block export** when contracts / joins are stale or unreviewed.

6. “Paste into Airflow? Then what?”

Discovery tools dump SQL into chat; ops still owns delivery.

What Que does: Hands off — signed artifacts, webhook to Airflow/Dagster, optional private runner, optional materialize in customer warehouse. Que does *not* become the warehouse or Airflow clone.

Do not claim yet: cleaning the data lake, hosting their warehouse, auto-accepting joins, or Que as system of record for production rows.

Part B — How Que works (product facts)

B.1 Roles — Viewer < Member < Admin < Owner

Action	Viewer	Member	Admin / Owner
View sources, schema, joins, jobs, lineage, settings (read)	Yes	Yes	Yes
Create / edit connections, sync	No	Yes (sync)	Yes (full connector admin)
Promote / reject joins, mapping-assist accept	No	Yes	Yes
Create / edit / run jobs, freeze, export, materialize	No	Yes	Yes
Chat / stitch session	No	Yes	Yes
Invites, roles, secrets, orchestrator / private-runner config, "run due schedules", billing checkout	No	No	Yes

Viewer = look only · **Member** = do the stitch work · **Admin/Owner** = people / billing / platform knobs.

B.2 Sync, joins, samples, jobs, schedules

2. After sync — full data or schema?

Only schema + optional capped samples in Que's metadata DB. Sync does *not* copy full tables into Que.

3. How PK/FK / joins are "proven"

Today it's a **trust ladder**, not automatic truth:

1. Declared keys / FKs from connectors when available
2. Heuristic / AI suggestions (scores + evidence)
3. Human **Promote** → accepted join

We do *not* require "10 matching sample rows → auto-trust." Samples support review; **Promote is the gate**. Export can block on unreviewed joins if that setting is on.

4. Samples — where, how many, who sees

- Live in Que metadata (`sample_values` on columns), usually ≤ 5 per column on sync
- Job dry-run preview uses up to ~10 illustrative rows from those samples
- Live validate pulls up to ~20 rows read-only from the source for that run (not stored as a Que lake)
- Scrubbing can tokenize PII
- Anyone with workspace access who can open schema/job UI can see what's stored (subject to scrub settings) — not "samples never leave the source" once sync-with-samples is enabled

5. "AI clean the whole data lake"

Out of scope. Que assists schema / mapping / job drafts (HITL). Row cleaning / lake transforms belong in the customer warehouse + their orchestrator.

6. “Make a new table” with joins — where is it created?

- Usually: SQL / dbt / JSON artifact (and optional GitHub PR) first
- Optionally: CTAS / VIEW in the **customer warehouse** via materialize (`confirm:true`)
- **Not** created as a permanent table inside Que

7. Job “produces a new table” — where is data stored / retained?

Path	Storage
Dry-run / live validate	Logs + tiny preview in <code>job_runs</code> — not a result table
Export / signed artifact	File / token download — schema/SQL/bundle, not a warehouse
Materialize	Customer warehouse object; Que keeps metadata audit only

Que is **not** the system of record for result rows.

8. How do they “see validated data”?

- **Que UI:** capped sample / live-validate preview on the job
- **Real table:** warehouse BI / SQL against the materialized object
- **Artifact:** download attested export

Not: a general Que “query the cleaned lake” API.

9. Schedule daily to “pull data from connectors”

Two different schedules:

- **Source schedule:** re-introspect schema (not full ETL into Que)
- **Job schedule:** re-run notebook dry_run or live validate (read-only), with retries

Not “pull all connector data into Que daily.” Warehouse refresh = materialize / external orchestrator.

10. Where does the job run?

- **Default:** Que API process (in-process ticker / request)
- **Optional:** private runner (customer callback) if execution target = private_runner
- **Airflow / Dagster:** Que only POSTs a signed webhook; they run the DAG — Que does not host Airflow

11. How another backend accesses “cleaned / job” outputs

- Customer warehouse (after materialize) with their own credentials
- Signed artifact URL (download export pack)
- Orchestrator webhook payload (ids / status / refs — not full tables)

There is no “query Que workspace for cleaned row datasets” product API.

B.3 Explicitly out of v1 product

- Que-hosted data lake / scratch warehouse
- Auto-accepted joins (no HITL)

- AI "clean the lake" / bulk row mutation in Que
- Que as system of record for production row data
- Full Airflow clone / full VPC BYOC agent (callback MVP only)
- Full daily ETL of all connector data into Que

Part C — Complete user guide (what you can do in the app)

Below is a screen-by-screen map of controls. Role hints: **V**=viewer ok · **M**=member+ · **A**=admin+ · **O**=owner where noted.

C.1 Login & account (`/login`)

Control	What you can do	Role
SIGN IN / CREATE ACCOUNT tabs	Switch between login and registration	—
EMAIL, PASSWORD	Credentials (password min 8 on register)	—
DISPLAY NAME, FIRST WORKSPACE NAME	On create account — names you and your first workspace	—
SIGN IN / CREATE ACCOUNT	Submit; lands on Workspace	—
Continue with SSO	OIDC start (when SSO is configured)	—
Dev demo buttons (Owner / Member / Viewer)	Local-only quick-fill credentials	—

Auth callback (`/auth/callback`): finishes SSO; on error use **BACK TO SIGN IN**.

C.2 App chrome & navigation

Control	What you can do
Que brand	Go to Workspace canvas
Menu	Mobile drawer: workspaces + all nav links
Workspace switcher	Pick another workspace; create workspace (name + CREATE); open canvas
Nav links	Workspace, AI Chat, Sources, Joins, Jobs, Lineage, Settings
?	Open onboarding roadmap dialog (SKIP / BACK / NEXT / DONE; optional deep-links)
Avatar / role chip	See your role; sign out → Login

C.3 Workspace — schema canvas (/workspace)

Purpose: See tables as a graph, filter relationships, Promote/Reject suggested joins on edges, start a stitch session, export a filtered schema snapshot.

Top bar filters & export

Control	What you can do	Role
Search (+ ⌘K / Ctrl+K)	Filter tables/columns on the canvas by name	V
Tools	On smaller screens: open filters + export + counts	V
Relationship type	Filter edges: All / Explicit / AI-inferred	V
Min confidence	Any / ≥50% / ≥70% / ≥85%	V
Source type	All, PostgreSQL, MongoDB, Snowflake, Databricks, SQL, MySQL, Excel, CSV, Kafka	V
Export PDF / PNG / JSON	Download or print the filtered schema view	V

Canvas toolbar & graph

Control	What you can do	Role
Hand / Select	Pan mode vs select mode	V
Zoom + / - / %	Zoom in/out; see zoom level	V
Snap	Toggle snap-to-grid	V
Layout	Auto-arrange nodes (write toast if viewer)	M
VIEW badge	Shows when you are read-only	V
STITCH SESSION	Open stitch dialog (≥2 sources, live API)	M
Stats	See Total Nodes / Relationships counts	V
Table nodes	Select, expand/collapse columns; drag to reposition	V / M drag
Relationship edges	Hover for confidence/evidence; Promote / Reject on suggested joins	M
MiniMap	Overview + pan viewport	V
Wheel / space-drag	Zoom / pan	V
Sync-drift banner	Notice schema changed; dismiss	V

Right sidebar (table selected)

Control	What you can do	Role
Close ×	Deselect table	V
Column pills	Focus a column	V
Refresh preview	Placeholder toast (samples coming soon in this panel)	V
Show lineage	Placeholder toast → use Lineage page	V
Save Changes	Create a stitch job from this table + neighbors → Jobs	M

Stitch Session dialog

1. Pick Source A and Source B
2. 1 · **INFER JOINS** — generate suggestions
3. Per suggestion: **PROMOTE** / **REJECT**
4. Enter job title → 2 · **CREATE JOB**
5. Optional 3 · **SHIP DBT PR**
6. **CLOSE**

C.4 Sources — connectors (/sources)

Purpose: Connect databases / warehouses / files / CRM; sync schema metadata; watch health; schedule re-introspect.

List view

Control	What you can do	Role
Filter sources...	Filter connector tiles by name	V
+ Add connector	Start create wizard	A
Install POC pack	Install Snowflake + Databricks demo fixtures	A
Source tiles	Open connection detail	V
Dashed Add tile	Same as Add connector	A

Connection health panel

Control	What you can do	Role
Healthy %	See workspace connector health summary	V
Needs attention list	See failing / stale connections	V
Re-auth / Fix	Jump to detail to fix credentials	A
Sync now	Re-introspect schema for that source	M
Open Workspace	Go to canvas	V
Join Review link	When healthy — go review joins	V

Create wizard

- **Category chips:** All, Databases, Warehouses, Files, CRM, Custom
- **Search** connectors by name
- **Pick a type:** PostgreSQL, MongoDB, Snowflake, Databricks, Excel, CSV, **BigQuery**, **Salesforce** (plus request-only S3 / Custom API)
- Request connector / Use CSV / Excel
- Continue to setup → fill form → Create connection or Upload & sync
- Warehouse / CRM types often offer **One-click fixture** vs **Live credentials**

Connection form fields (by type)

Type	Fields you fill
Common	Name, Description; Type locked when editing
PostgreSQL	Host, Port, Database, Schema, User, Password
MongoDB	URI, Database
Databricks (live)	Host, Warehouse ID, Token, Catalog, Schema
Snowflake (live)	Account, Warehouse, Database, Schema, User, Token, Password
BigQuery (live)	Project id, Dataset, Location, OAuth access token
Salesforce (live)	Instance URL, Access token
Excel / CSV	File upload, Table names, Mapped files JSON

Detail view actions

Control	What you can do	Role
← Back to Sources	Return to list	V
Edit connection fields	Update credentials / config (fieldset disabled if not admin)	A
Sync schedule	Off / Hourly / Daily re-introspect	A
Save	Persist connection + schedule	A
Sync Schema	Pull schema (+ capped samples if policy allows)	M
Delete	Remove connection	A
Retry sync / Update credentials	Recover from error banner	M / A
Open Workspace	See synced tables on canvas	V

C.5 Join Review + mapping assist (/joins)

Purpose: Inbox for suggested joins and column rename suggestions — HITL gate before jobs trust them.

Control	What you can do	Role
Re-run inference	Rescan and refresh join suggestions	M
Suggest mappings	Run mapping assist; fill rename queue	M
Open Workspace	See joins on canvas	V
Status filters	Pending / Accepted / Rejected / All	V
List item	Select to see evidence / scores in detail	V
Promote / Reject	Accept join into workspace truth, or reject	M
Rename: Accept / Reject / Dismiss	HITL on column alias suggestions (intent only)	M

C.6 AI Chat (/chat)

Purpose: Schema-only assistant — list tables, describe, suggest joins, draft SQL / jobs. Does *not* clean your lake.

Control	What you can do	Role
Skills	Show/hide skill chips	V
Reindex	Rebuild schema embeddings for RAG	M
Stop / Clear	Abort generation / clear thread	M / V
Skill chips	Insert or run: <code>/list</code> <code>/describe</code> <code>/joins</code> <code>/suggested</code> <code>/sql</code> <code>/job</code> <code>/diff</code> <code>/privacy</code> <code>/help</code>	M
Empty quick chips	One-click List / Suggested / Diff / Help	M
Mention chips ×	Remove @table focus from context	M
Generate SQL / Explain table / Visualize joins / Regen	Shortcut actions on assistant messages	M
Composer	Type question; Enter to send; @ and / autocomplete	M
Attach / @ / / / Mic	Files, mention tables, skills, voice	M
Model select	Pick generation model	M
Send	Ask the assistant	M
Copy / +1 / -1	Copy answer; thumbs feedback	M
Save job	If assistant drafted a job — create it in Jobs	M
Sidebar: Refresh, Filter tables, expand, click/drag	Browse schema; add @table / @column context	V / M

Viewers see a read-only composer and “viewer cannot chat” messaging.

C.7 Jobs — notebook → deploy (/jobs)

Purpose: Turn reviewed stitch into a notebook job; dry-run / live-validate; freeze contract; export attested pack; optionally materialize in customer warehouse; schedule re-runs.

Monitor list

Control	What you can do	Role
Filter...	Filter job cards by title	V
↻ Refresh	Reload jobs	V
+ New Job	Open create dialog	M
Job cards	Open job editor	V
Empty: Create manually / Draft in AI Chat	Start from scratch or go to Chat	M
Run history	See past runs; click title to open job	V
Live Stream ▶ / II	Pause/resume log stream	V
Search logs...	Filter streaming log lines	V
FAB + (mobile)	New job	M

New Job modal

Title; optional table chips; **CANCEL** / **CREATE** (member+).

Job editor — header

Control	What you can do	Role
← back	Return to monitor	V
Run Test	Execute notebook in selected mode	M
Mode: Dry-run ≤10	Illustrative preview from capped samples (~10 rows)	M
Mode: Validate ≤20	Live read-only pull ~20 rows from sources for this run	M
Commit	Save notebook edits	M
Deploy →	Jump to Deploy tab	M
Tabs: Notebook Results Deploy	Switch editor surfaces	V

Notebook tab

Control	What you can do	Role
+ SQL / + Markdown	Add cells	M
Run cell	Run one cell	M
Discard	Revert unsaved cell edits	M
Cell kind / title / editor	Edit cell content	M
↑ ↓ Run Delete	Reorder, run, remove cells	M
Process / Output / Logs + Expand	Inspect cell run detail	V
View results / Open results	Jump to Results tab	V

Results tab

- See capped sample or live-validate preview
- Run again to re-execute

Deploy tab — what you can ship

Control / panel	What you can do	Role
Schedule & retries	Off / Hourly / Daily; run mode dry or live; execution Que vs Private runner; max retries; retry delay; Save schedule	M
Drift Acknowledge	Ack open drift so deploy can proceed when required	M
Contract freeze	Freeze / Re-freeze snapshot; Validate; Open Joins	M
Deploy destination	Choose File/PR vs Customer warehouse	M
Mark ready	Mark job ready for export	M
Open dbt PR	Open GitHub PR with attested dbt (when GitHub configured)	M
Export dbt bundle	Download attested pack	M
Download SQL / JSON	Download artifacts	M
Mint signed URL	Time-limited download link for another system	M
Per-file Download	Download individual export files	M
Materialize	Pick connection; view vs table; schema; name; replace checkbox; confirm checkbox; Create view/table in <i>customer</i> warehouse	M
Links	Join Review, Verify attestation, Lineage, Workspace	V

C.8 Lineage (/Lineage)

Purpose: See the metadata path Sources → joins → job → export / materialize.

Control	What you can do	Role
Refresh	Reload lineage paths	V
Job path list	Select a path to inspect stages	V
Open Deploy	Jump to that job's Deploy tab	V
Stage links	Navigate to Sources, Join Review, Verify	V
GitHub PR URL	Open external PR if present	V

C.9 Settings (/settings)

Purpose: People, secrets, AI/export policy, ops panels (drift, schedules, orchestrator, private runner, billing), audit.

Members & invites

Control	What you can do	Role
Invite Member	Open invite modal (email + role)	A
Member search	Filter member registry	V
Role <select>	Change member role (not self / last owner); owner option if you are owner	A / O
Remove	Remove member from workspace	A
Revoke	Cancel pending invite	A

Always-visible strips

- API Keys: **View all** / **Generate New Key**
- Env: + opens advanced
- Stats: Connections / Tables / Relations / Jobs
- Footer: **Manage Billing** / **Upgrade Plan**
- Show/Hide policy, BYOK & AI settings

Ops & platform panels

Panel	What you can do	Role
SSO	See SSO readiness status	V
Usage	See soft-limit bars (seats / usage)	V
Drift alerts	Refresh ; Send test alert ; Ack ; Re-notify	Ack any; test/re-notify A
Scheduled sync	Run due now ; Refresh — fire due source schema syncs	A
Scheduled jobs	Run due now ; Refresh — fire due job schedules	A
External orchestrator	Webhook URL; Kind (generic / Airflow / Dagster); Enabled; Save; Rotate secret; Test ping	A
Private runner	Runner URL; Enabled; Save; Rotate secret	A
Billing	Seats; Upgrade Checkout; Manage Portal; Refresh	A
Export attestations	Open verify page; Refresh; download Attestation / Verify pack	V
Signed artifacts	Refresh; Revoke a signed URL	Revoke A
Audit log	Refresh and read who did what	V

Advanced policy toggles (admin edits)

- Include samples · Scrub samples
- Infer joins on sync · Databricks query-history join assist
- Prefer LLM chat · Include docs in RAG
- **Block export on high drift** · Block dbt PR on column drift · **Block export on unreviewed joins**
- Emit contract events · Contract webhook URL
- Drift alerts enable / high-only · Drift webhook · Alert emails
- Default model · RAG top-K
- **RESET / SAVE · RE-RUN JOIN INFERENCE** (member+)

BYOK & dbt GitHub

- OpenAI / Anthropic key: save / clear
- GitHub: owner, repo, base branch, dbt path, token
- Reindex schema + docs

C.10 Attestation verify — public (/verify)

Purpose: Anyone (no login) can paste attestation JSON and check the HMAC signature.

Control	What you can do
Textarea	Paste attestation JSON from an export
Verify signature	See valid/invalid + fingerprint / policy / algorithm
Clear	Reset form
Links	Settings / Sign in

Part D — Typical end-to-end journey (what a member actually does)

1. **Admin** invites people; sets policy (samples/scrub/block-on-unreviewed).
2. **Admin** adds connectors on Sources (Postgres, Mongo, Excel, Snowflake, Databricks, BigQuery, Salesforce...); optional sync schedule.
3. **Member** clicks **Sync Schema** — schema (+ capped samples) land in Que metadata.
4. Open **Workspace**: filter by source/confidence; review edges; **Promote** / **Reject** (or use **Joins** inbox + **Suggest mappings**).
5. Optional: **AI Chat** to describe tables / draft SQL / save a job.
6. Or **STITCH SESSION** / sidebar **Save Changes** → create job.
7. In **Jobs**: edit notebook → **Run Test** (dry-run or live validate) → see tiny preview.
8. **Deploy**: Freeze contract → Export attested pack / open dbt PR / mint signed URL — and/or **Materialize VIEW/table** in *their* warehouse.
9. Optional: schedule job re-validate; wire Airflow webhook / private runner in Settings.
10. Use **Lineage** + **/verify** to show the path and prove the export.

Part E — What's newly built (user-level)

Feature	Where in the app	What you can do
BigQuery connector	Sources → Add → Warehouses	Fixture or live project/dataset/token; sync schema into canvas
Salesforce connector	Sources → Add → CRM	Fixture or live instance URL + token; sync object schema
Job schedules & retries	Jobs → Deploy; Settings → Scheduled jobs	Hourly/daily dry-run or live validate; retries; admin “run due now”
Orchestrator webhook	Settings → External orchestrator	Point Que at Airflow/Dagster/generic; rotate secret; test ping
Mapping assist	Joins → Suggest mappings	Review column-alias suggestions; Accept / Reject / Dismiss
Private runner	Settings + Jobs schedule execution target	Customer callback MVP so jobs can target private_runner
Soft seat billing	Settings → Billing	Seats, Stripe checkout / portal (when keys configured)
Materialize	Jobs → Deploy → Customer warehouse	Create VIEW/table in their warehouse with explicit confirm
Attestation + signed artifacts	Jobs export; Settings; /verify	Prove export integrity; mint/revoke signed download URLs
Drift + contract freeze	Workspace banners; Jobs Deploy; Settings policy	Ack drift; freeze contracts; block bad exports via policy
Connector health + source schedules	Sources list + detail	See health %; Sync now; Hourly/Daily schema re-introspect

Buyer sentence: Que solves cross-source stitch correctness + safe AI + attested handoff — getting messy multi-system schemas to a human-approved, exportable job — without taking custody of the lake.